

1. The 8 Primitive Data Types (The Building Blocks)

Primitives are the most basic data types in Java. They store raw, plain values directly in memory. They are not objects, they have no methods, and they are incredibly fast.

Type	Size	Default	Range / Description
byte	8-bit	0	-128 to 127. Used for saving memory in large arrays.
short	16-bit	0	-32,768 to 32,767. Rarely used today.
int	32-bit	0	Standard choice for whole numbers.
long	64-bit	0L	Huge numbers (e.g., database IDs, milliseconds).
float	32-bit	0.0f	Decimal numbers (requires f suffix).
double	64-bit	0.0d	Standard choice for precise decimals.
char	16-bit	\u0000	A single Unicode character (e.g., 'A').
boolean	1-bit	false	true or false logic flags.

2. What are Wrapper Classes?

A Wrapper Class is a full Java Object that encapsulates (wraps) a single primitive value. Every primitive has a corresponding Wrapper Class.

- byte -> Byte
- short -> Short
- int -> Integer
- long -> Long
- float -> Float
- double -> Double
- char -> Character
- boolean -> Boolean

3. Primitives vs. Wrapper Classes: Core Differences

Feature	Primitive (int)	Wrapper Class (Integer)
Nature	Raw binary value.	Full Java Object.
Memory Location	Stored on the Stack (fast access).	Stored on the Heap (slower access).
Nullability	Cannot be null. Defaults to 0 or false.	Can be null. Defaults to null.
Performance	Extremely fast. Zero overhead.	Slower. Carries Object header memory overhead.
Methods	None. Cannot call <code>.toString()</code> or <code>.equals()</code> .	Contains utility methods (e.g., <code>Integer.parseInt()</code>).

4. Why Do We Use Wrapper Classes? (The "Why")

If primitives are faster and use less memory, why do Wrapper Classes exist?

1. **The Java Collections Framework:** Data structures like `ArrayList`, `HashMap`, and `HashSet` can **only** store Objects. You cannot create an `ArrayList`. You must use `ArrayList`.
2. **Handling Missing Data (Nulls):** In a database, an "Age" column might be empty (null). A primitive `int` age cannot hold null; it defaults to 0 (which implies the user is a newborn). An `Integer` age can hold null, accurately representing "data missing."
3. **Utility Methods:** Wrappers provide built-in methods for data conversion. Converting a `String` to a number requires the wrapper: `int count = Integer.parseInt("50");`
4. **Generics:** Java Generics () only work with Objects.

5. Why Do We Still Use Primitives?

1. **Performance & Math:** Mathematical operations (+, -, *, /) are significantly faster with primitives because they skip the Object overhead.
2. **Garbage Collection:** Wrapper classes create Objects on the Heap. Creating millions of `Integer` objects (e.g., in a massive loop) triggers the Garbage Collector, which freezes the application briefly to clean up memory. Primitives live on the Stack and disappear instantly when the method finishes.

6. Autoboxing and Unboxing

Java automatically converts between primitives and their Wrappers behind the scenes to save you time.

- **Autoboxing:** Converting a primitive to a Wrapper automatically.
 - `Integer myNum = 10;` (Java automatically wraps the 10 into an Integer object).
- **Unboxing:** Converting a Wrapper back to a primitive automatically.
 - `int rawNum = myNum;` (Java automatically extracts the int from the object).

Interview Vault: Advanced & Viral Questions

Q: "What will happen when this code runs?"

Java

```
Integer a = null;  
int b = a;  
System.out.println(b);
```

Answer: It throws a **NullPointerException** at runtime.

The Trap: This tests your knowledge of Unboxing. Java tries to automatically extract the primitive value from the Integer object a to assign it to b. Because a is null, there is no object to extract from, causing a crash.

Q: "Explain the Integer Cache mechanism and predict the output of this code."

Java

```
Integer num1 = 127;  
Integer num2 = 127;  
System.out.println(num1 == num2);  
  
Integer num3 = 128;  
Integer num4 = 128;  
System.out.println(num3 == num4);
```

Answer: It prints **true** then **false**.

The Trap: When using `==` on Objects, Java checks if they share the exact same memory address. To save memory, Java caches Integer objects for values between -128 and 127. Because 127 is in the cache, num1 and num2 point to the exact same object in memory. 128 is outside the cache, so the JVM creates two completely separate objects on the Heap. (Note: Always use `.equals()` to compare Wrapper classes!).

Q: "Method Overloading Priority: Which method will be executed by the JVM?"

```
public void calculate(long num) {  
    System.out.println("Long method");  
}  
  
public void calculate(Integer num) {  
    System.out.println("Integer Wrapper method");  
}  
  
// Main method execution:  
int myPrimitive = 5;  
calculate(myPrimitive);
```

Answer: It prints "**Long method**".

The Trap: This tests how the JVM prioritizes operations. When passing an int, the JVM has two choices: **Widen** it to a long, or **Autobox** it to an Integer. In Java, **Widening always beats Boxing**. The JVM will choose to upcast the int to a long primitive rather than creating a new Integer object.

Q: "How much more memory does an Integer object take compared to an int primitive?"

Answer: An int primitive takes exactly **4 bytes** (32 bits). An Integer wrapper class typically takes **16 bytes** (12 bytes for the Object header metadata + 4 bytes for the actual integer payload). Therefore, an Integer consumes about **4x more memory** than a primitive int.